



RAPPORT DE PROJET

Bibliothèque Numérique DIT

Examen DevOps — Dakar Institute of Technology

Réalisé par :

DIALLO Salou

OMAR Moussa

DIA Adj Ndioba

DIA Amadou Daouda

DIACK Mariam Bocar

DIAKITE Mamadou Daye

Filière : Master 1 Intelligence Artificielle

Date : Juillet 2026

1. Présentation du projet

Le Dakar Institute of Technology (DIT) gère actuellement sa bibliothèque académique de manière manuelle, ce qui pose plusieurs problèmes : difficulté de suivi des livres, absence de statistiques fiables, gestion inefficace des emprunts et manque d'accès numérique pour les étudiants.

Ce projet propose une plateforme web moderne, la « Bibliothèque Numérique DIT », permettant de digitaliser l'ensemble des opérations de la bibliothèque : gestion des livres, des utilisateurs et des emprunts. L'application repose sur une architecture microservices, conteneurisée avec Docker, et déployée automatiquement via un pipeline CI/CD Jenkins.

Objectifs du projet

- Remplacer la gestion papier/manuelle par un système numérique centralisé.
- Permettre la recherche rapide d'un livre par titre, auteur ou ISBN.
- Distinguer les types d'utilisateurs (Étudiant, Professeur, Personnel administratif).
- Suivre les emprunts en cours et détecter automatiquement les retards.
- Automatiser le déploiement de l'application à chaque mise à jour du code.

2. Architecture du système

L'application est construite selon une architecture microservices : chaque fonction métier (livres, utilisateurs, emprunts) est isolée dans un service indépendant, avec sa propre base de code, son propre Dockerfile, et communique avec les autres via des appels API REST en HTTP/JSON.

Composant	Port	Rôle
Frontend	3000	Interface web (HTML/CSS/JS) servie par Nginx, point d'entrée utilisateur.
Service Livres	8000	API FastAPI gérant le CRUD des livres et la recherche.
Service Utilisateurs	8001	API FastAPI gérant les comptes et profils utilisateurs.
Service Emprunts	8002	API FastAPI gérant les emprunts, retours et retards.
PostgreSQL	5432	Base de données relationnelle partagée, un schéma par service.

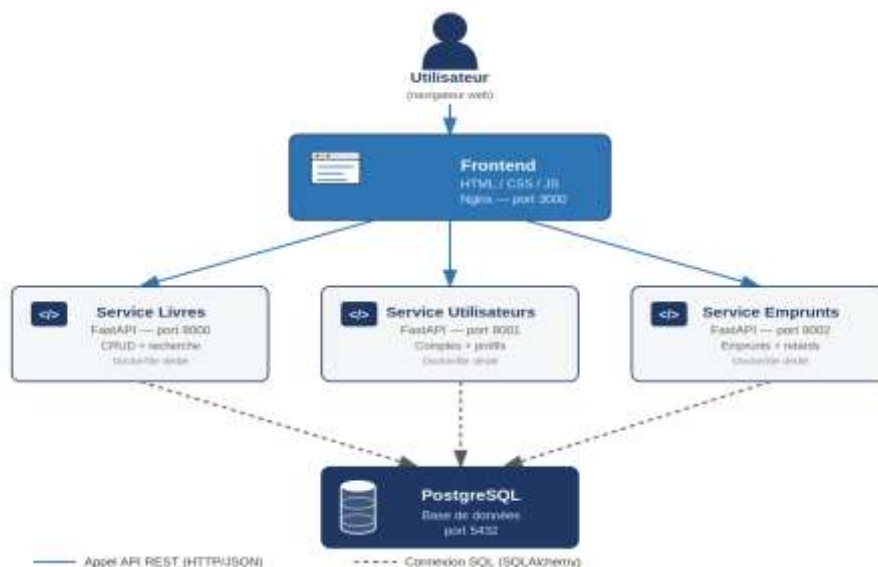


Figure 1 — Schéma de l'architecture microservices : flux d'appels API (traits pleins) et connexions à la base de données (traits pointillés).

Chaque microservice backend possède sa propre image Docker (construite depuis un Dockerfile dédié) et communique avec la base PostgreSQL via une URL de connexion (SQLAlchemy). Le frontend, servi par un conteneur Nginx, effectue des appels fetch() en JavaScript vers les 3 APIs backend. L'ensemble est orchestré par Docker Compose, qui définit le réseau interne, les volumes persistants et l'ordre de démarrage des services.

Flux de communication simplifié :

- Le navigateur charge le frontend (port 3000).
- Le frontend appelle directement les APIs Livres (8000), Utilisateurs (8001) et Emprunts (8002).
- Chaque service interroge sa base de données PostgreSQL (port 5432) via SQLAlchemy.
- Les services sont indépendants : aucun n'appelle directement un autre microservice.

3. Description des microservices

3.1 Service Livres (port 8000)

Développé en FastAPI avec SQLAlchemy pour la persistance PostgreSQL. Il expose les endpoints suivants :

Méthode	Route	Description
GET	/livres	Liste tous les livres enregistrés.
POST	/livres	Ajoute un nouveau livre (titre, auteur, ISBN).
PUT	/livres/{id}	Modifie les informations d'un livre existant.
DELETE	/livres/{id}	Supprime un livre par son identifiant.
GET	/livres/recherche?q=	Recherche un livre par titre, auteur ou ISBN.

3.2 Service Utilisateurs (port 8001)

Gère les comptes des trois types d'utilisateurs : Étudiant, Professeur et Personnel administratif.

Méthode	Route	Description
GET	/utilisateurs	Liste tous les utilisateurs.
POST	/utilisateurs	Crée un nouvel utilisateur (avec vérification d'unicité de l'email).
GET	/utilisateurs/{id}	Consulte le profil détaillé d'un utilisateur.
PUT	/utilisateurs/{id}	Modifie les informations d'un utilisateur.
DELETE	/utilisateurs/{id}	Supprime un utilisateur.

3.3 Service Emprunts (port 8002)

Gère le cycle de vie d'un emprunt, de la création au retour, ainsi que la détection automatique des retards.

Méthode	Route	Description
POST	/emprunts	Enregistre un nouvel emprunt (livre, utilisateur, date de retour prévue).
PUT	/emprunts/{id}/retour	Marque un emprunt comme retourné (horodatage automatique).
GET	/emprunts	Retourne l'historique complet des emprunts.
GET	/emprunts/utilisateur/{id}	Liste les emprunts d'un utilisateur donné.

GET	/emprunts/retards	Retourne les emprunts en retard (date dépassée et non retournés).
-----	-------------------	---

Les trois services partagent la même base PostgreSQL et créent automatiquement leurs tables au démarrage (SQLAlchemy Base.metadata.create_all). Chaque service attend activement que la base soit disponible avant de démarrer (fonction wait_for_db), afin d'éviter les erreurs de connexion lors du lancement simultané des conteneurs.

4. Explication du pipeline CI/CD

Le déploiement est automatisé via un pipeline Jenkins défini dans un Jenkinsfile à la racine du projet. Il se compose de trois étapes principales (stages) exécutées séquentiellement :

#	Étape	Action réalisée
1	Récupération du code	checkout scm — récupère la dernière version du code source depuis le dépôt GitHub configuré dans le job Jenkins.
2	Build des images Docker	docker compose build — reconstruit les images Docker de chaque service (livres, utilisateurs, emprunts, frontend) à partir de leurs Dockerfile respectifs.
3	Déploiement	docker compose up -d — démarre (ou redémarre) l'ensemble des conteneurs en arrière-plan avec la configuration définie dans docker-compose.yml.

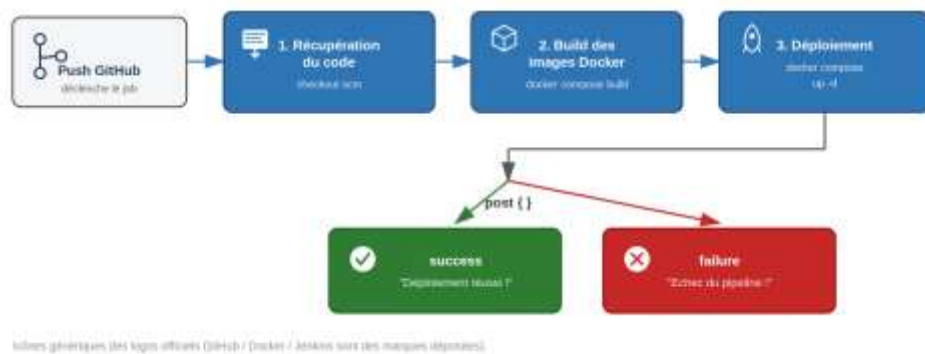


Figure 3 — Flux du pipeline Jenkins : les trois étapes s'exécutent séquentiellement, suivies du bloc post qui affiche un message de succès ou d'échec selon le résultat.

Un bloc post gère le résultat final du pipeline : un message de succès s'affiche si toutes les étapes se sont bien déroulées, ou un message d'échec en cas de problème à n'importe quelle étape.

Avantages de cette automatisation :

- Élimination des étapes manuelles répétitives (clone, build, déploiement).
- Déploiement reproductible et identique à chaque exécution.
- Historique des builds consultable dans l'interface Jenkins (succès/échec, logs détaillés).
- Base extensible pour ajouter plus tard des tests automatisés ou une analyse de qualité de code (SonarQube).

5. Captures d'écran de l'application

Tableau de bord, offrant une vue d'ensemble en temps réel du catalogue, des utilisateurs, des emprunts en cours et des retards à traiter :

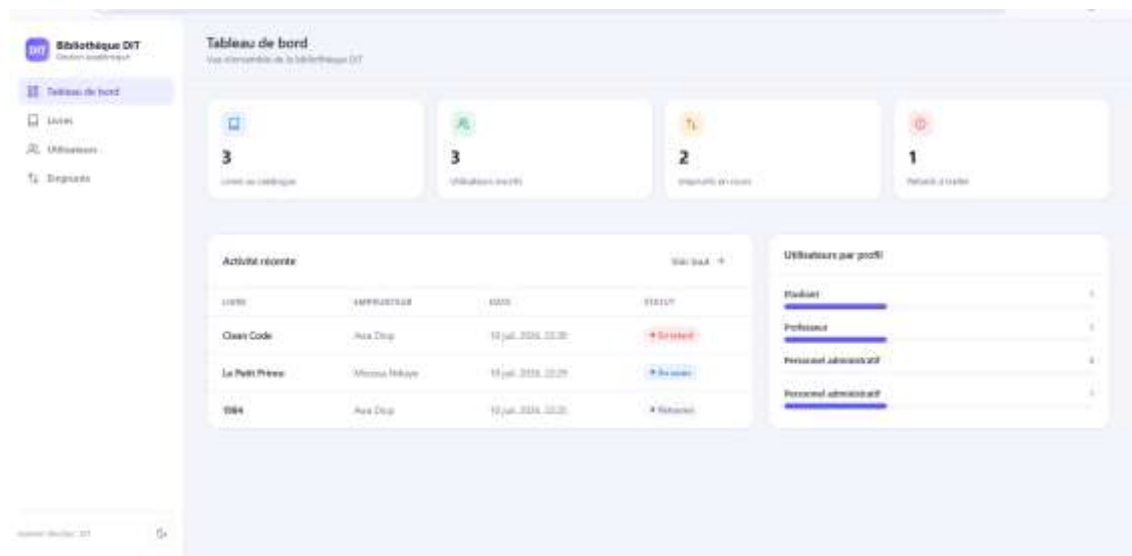


Figure 4 — Tableau de bord : statistiques globales et répartition des utilisateurs par profil.

Gestion des livres : liste du catalogue avec recherche, ajout, modification et suppression :



Figure 5 — Page « Livres » : catalogue avec statut de disponibilité et actions Éditer / Supprimer.

Gestion des utilisateurs, avec distinction des trois profils (Étudiant, Professeur, Personnel administratif) :



Figure 6 — Page « Utilisateurs » : liste avec type de profil, consultation (icône œil), édition et suppression.

Suivi des emprunts : historique complet avec détection automatique des retards (statut « En retard » affiché en rouge) :

TYPE	EMPRUNTEUR	DATE D'EMPRUNT	RETOUR À	STATUT	
Clarin Code	Alex Delp	10 juil. 2024, 12:08	17 juil. 2024, 00:00	En retard	Retourner
Le Petit Prince	Thomas Pélage	10 juil. 2024, 12:08	10 juil. 2024, 00:00	En cours	Retourner
1984	Alex Delp	10 juil. 2024, 12:08	17 juil. 2024, 00:00	Retourné	

Figure 7 — Page « Emprunts », onglet Historique : un emprunt en retard, un en cours et un retourné.

Exécution du pipeline Jenkins : les 5 étapes (Consultez SCM, Récupération du code, Créer des images Docker, Déploiement, Actions sur la publication) se sont déroulées avec succès :



Figure 8 — Build Jenkins #8 : pipeline exécuté avec succès, message « Déploiement réussi ! » confirmé dans les logs.